Pointers, increment and decrement operators

Sudeep K S, Mahindra University (October 2026)

Remember how we swapped the values of two int variables. The code looked like this:

```
int temp = a;
a = b;
b = temp;
```

What if we write a function to do the same, with the same logic, as below?

```
// interchange the values in variables a and b
void swap(int a, int b)
{
   int temp = a;
   a = b;
   b = temp;
}
```

Check what actually happens if you call this function from main. It turns out that the values of a and b do not change even after a call like `swap(a, b);` though the function swaps the values internally.

You can verify this by printing values of a and b after this call in `main()`. This happens because only the values of a and b are passed to the function, not the 'locations' where we need the values changed. The values of a and b changed inside the function, because what we mean by 'a' inside the function is not the same as the variable 'a' at the point where we call it from.

A solution to this is to pass 'pointers' into the function. A **pointer** specifies the location or the address in the machine's memory where the variable resides. See the figure below.

	Pointer (or **location** or **address**)	Content at the memory
a	1052 (location reserved for variable a)	5 (value of a)
b	2048 (location reserved for variable b)	10 (value of b)

When we declare a variable as `int a;` what actually happens is that the machine **allocates space** that would fit an integer variable, and link the name 'a' to that location. (Similarly, it assigns enough space for a character if it was a `char a;` enough space for float if it is `float a.`)

When we define a value with an assignment statement like `a = 5;` it writes the value 5 at the location reserved for variable a.

When we print the value of a or try to use 'a' in any arithmetic or logical expression anywhere in the program afterwards, it looks at that location and gets the value of a from there.

So, if we have a way to send the locations of these variables into the swap function and manipulate the values at those locations, it would work correctly. Let us see how.

We can get the 'location of a' (or pointer to a) by '&a' in the program. Similarly, if we have the pointer (location) in **p**, we can get the value of the variable at that location by '*p'.

In the above figure, we have the location and value of variables a and b. A pointer variable p that points to an integer can be declared as `int *p;` or `int* p;`

We can assign value to such a variable as `p = &a;`

(In this example, this would give value '1052' to p, and it holds details about the location of a.)

Now, we can rewrite the swap function as follows:

```
// interchange the values in a and b (pointed to by p1 and p2)
void swap(int *p1, int *p2) // *p1 is a and *p2 is b
{
   int temp;
   temp = *p1;
   *p1 = *p2;
   *p2 = temp;
}
```

We can call this function as

```
swap(&a, &b);
```

This would swap the values of variables a and b. You may verify it using a printf statement.

Remember this:

- Given a variable a, get the location of a (pointer to a): `&a`
- Given a pointer variable p, get the actual value of the variable: `*p`

Another example is a function that finds sum of two variables by taking pointers:

```
int sum(int *p1, int *p2) // *p1 is a and *p2 is b
{
   return (*p1 + *p2);
}
```

This would work but it is not really necessary, because we just need the sum as the return value. The below function would be sufficient.

```
int sum(int a, int b)
{
   return (a + b);
}
```

However, if we need to assign the sum value to the variable a, we can make use of pointers.

```
void sassign_sum(int *p1, int *p2) // *p1 is a and *p2 is b
{
   *p1 = *p1 + *p2; // a gets the value a + b
}
```

Or just this much, by passing the pointer only for the first variable:

```
void assign_sum(int *p1, int b) // *p1 is a
{
   *p1 = *p1 + b; // a gets the value a + b
}
```

Good to know: When we pass an array variable into a function, what actually gets passed is the location of the first element in the array. So, the two functions given below are identical.

```
void add_a_to_first_element(int marks[], int a)
{
   marks[0] = marks[0] + a; // a is added to the first element
}
```

is the same as

```
void add_a_to_first_element(int marks[], int a)
{
   *marks = *marks + a; // a is added to the first element
}
```

Also, `marks + i` would give the location of the ith element in the array, `marks[i]`.

In other words, `*(marks + i)` is same as `marks[i]`; and `&marks[i]` is nothing but `marks + i`.

Go through the character arrays part in the previous set of notes once again, it makes use of this.

For example, recollect that

```
while (name[i] != '\0') {
   printf("name[%d] is %c\n", i, name[i]);
   i++;
}
```

is the same as

```
while (*(name + i) != '\0') {
   printf("%d th character is %c\n", i, *(name + i));
   i++;
}
```

Difference between ++a and a++; --a and a--

It is important to understand the difference between ++a and a++; and --a and a--.

While the first one (++a) adds 1 to a first and uses that value, 'a++' uses the current value of a and then adds 1 to a (to the variable at the location of a).

So, if we have

```
int a = 5;
printf("a is %d", ++a);
```

it would print "`a is 6`".

However, if it was

```
int a = 5;
printf("a is %d", a++);
```

it would print "`a is 5`".

In both the cases, if we have a statement

```
printf("a is %d", a);
```

after this, it will print "`a is 6`" only.

Note that '--' operator works the same way. If used as prefix, it uses the value after deducting 1. Else it uses the current value of the variable and then deducts 1.